Deploy a static website with GitHub Pages

A step-by-step runbook that takes you from an empty folder to a public HTTPS website. Every command is explained, every output is shown, and every failure has a fix.

Project 1 of 3 | About 30 minutes | Cost: free | Level: beginner



Know what you built. GitHub Pages serves your files from GitHub’s own infrastructure. You do not install or run Nginx in this project. That comes in Project 2 with Docker. In an interview, describe this as static hosting.

My implementation

The steps below use example names so anyone can follow along. These are the real details of this project, checked on October 30, 2026.

Repository
github.com/bashir-shaikh/nginx-server-visualization

Live website
bashir-shaikh.github.io/nginx-server-visualization

Hosting

GitHub Pages serving a static site from the main branch and the root folder, with HTTPS enforced (see Figure 1). No Nginx server was run for this project.

What the live page shows

The page title is “Nginx-Verse | My First DevOps Project”. It has a heading, an “About This Project” section about learning Git, GitHub and web deployment, a “Deployment Status” line, and a footer reading “Built with HTML and CSS”. See Figure 2 in step 8.

One fix to make. The Deployment Status section still says the next step is to deploy with GitHub Pages. The site is already deployed, so update that text to say it is live. Then commit and push the change, which is a good practice run of step 9.

Before you start

Four things, all free. Example names are used throughout: replace `your-username` with your GitHub username and keep `nginx-verse-site` as the repository name.

You need	Why	Check it
GitHub account	Hosts the repository and the website	Sign in at github.com
Text editor	Writes the HTML and CSS files	VS Code, Notepad or any plain-text editor
Web browser	Previews and verifies the site	Any current browser
Git (command-line route only)	Sends files to GitHub	`git --version`

The nine steps

Follow them in order. Each step ends with a check, so you always know it worked before you move on.

1 Create the repository

GitHub Pages publishes files from a repository, so you need one first.

Do this in your browser

1. Click **+** at the top right of GitHub, then **New repository**.
2. Repository name: `nginx-verse-site`.
3. Visibility: **Public**.
4. Tick **Add a README file**. This creates the `main` branch.
5. Click **Create repository**.

Naming rule: a repository called `your-username.github.io` is served at `https://your-username.github.io/`. Any other name is served at `https://your-username.github.io/repository-name/`. This guide uses the second form.

Success looks like

The page `github.com/your-username/nginx-verse-site` opens and lists `README.md`.

> **If it fails: Name already exists**
> You own a repository with that name. Pick another name and use it everywhere below.

2 Create the project folder

The files must exist on your computer before you can upload them. Keep the layout simple to avoid path errors.

Run on your computer (Linux, macOS, or Git Bash on Windows). No administrator rights needed.

terminal Copy

```
mkdir nginx-verse-site
cd nginx-verse-site
mkdir css
touch index.html css/style.css
```

`mkdir` creates a folder. `cd` moves into it so later commands act there. `touch` creates empty files. In Windows PowerShell use `New-Item index.html, css\style.css` instead of `touch`.

Check your work

terminal Copy

```
ls -R
```

expected output

```
.:
css  index.html

./css:
style.css
```

Success looks like

Two files exist: `index.html` and `css/style.css`. The name `index.html` matters because web servers show it by default at the site root.

> **If it fails: File saved as index.html.txt**
> Windows hid the extension. Turn on “File name extensions” in File Explorer and rename the file.

3 Write index.html

This is the page visitors see. Without it, Pages has nothing to show at your address.

Paste into index.html and save

index.html Copy

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Nginx-Verse on GitHub Pages</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header><h1>Welcome to Nginx-Verse</h1></header>
  <main><p>This static website is deployed using GitHub Pages.</p></main>
  <footer><p>Project 1 of the Nginx-Verse roadmap</p></footer>
</body>
</html>
```

`<!DOCTYPE html>` declares HTML5. `lang="en"` helps screen readers. `charset="UTF-8"` prevents garbled text. The `viewport` line makes the page fit phones. `<link rel="stylesheet" href="css/style.css">` loads the stylesheet by a relative path, which is case-sensitive. `header`, `main` and `footer` label the page sections.

Success looks like

Double-click `index.html`. The heading and paragraph show in your browser.

If it fails: Blank page

The text went into the wrong file or a tag is missing. Compare with the code above.

4 Write the stylesheet

Styling proves that supporting files are published and linked correctly, not only the HTML.

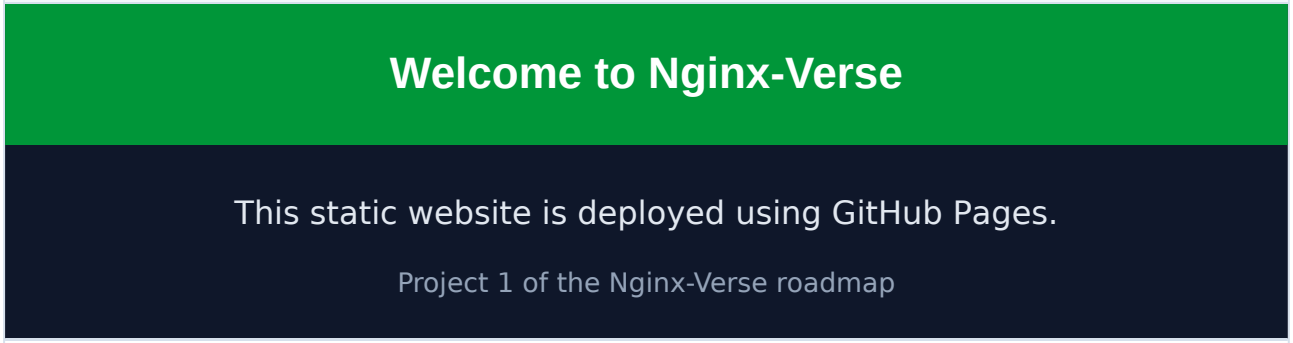
Paste into css/style.css and save

css/style.css Copy

```
body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  background-color: #0f172a;
  color: #e2e8f0;
  text-align: center;
}
header {
  padding: 2rem 1rem;
  background-color: #009639;
}
main { padding: 2rem 1rem; }
footer { padding: 1rem; color: #94a3b8; }
```

`margin: 0` removes the default page margin. The colours are hex codes. `padding` adds space inside an element. `rem` scales with the root font size. `text-align: center` centres the text.

https://your-username.github.io/nginx-verse-site/



What a successful result looks like (illustration built from the example files in this guide)

Success looks like

Refresh the local preview. A green header band appears on a dark background.

If it fails: No styling appears

The `href` in index.html does not match the real path. Check spelling and letter case.

5 Upload the files to GitHub

Pages can only publish what is inside the repository.

Web upload | Git command line

1. Open the repository, click **Add file**, then **Upload files**.
2. Drag in `index.html` and the `css` folder.
3. Commit message: `Add website files`.
4. Select **Commit directly to the main branch**, then **Commit changes**.

A commit is a saved snapshot with a message describing the change.

```
nginx-verse-site/
├── README.md
├── index.html
└── css/
    └── style.css
```

Success looks like

Refresh the repository page. `index.html` and the `css` folder are listed with your commit message.

If it fails: Authentication failed

Use a personal access token or SSH, not your account password.

6 Turn on GitHub Pages

Pages stays off until you tell it which branch and folder to publish.

1. In the repository, open **Settings**, then **Pages**.
2. Under **Build and deployment**, set **Source** to **Deploy from a branch**.
3. Branch: `main`. Folder: `/ (root)`.
4. Click **Save**.

Use `/docs` only if your files live in a `docs` folder. “GitHub Actions” as a source is for custom builds and is not needed for plain HTML.

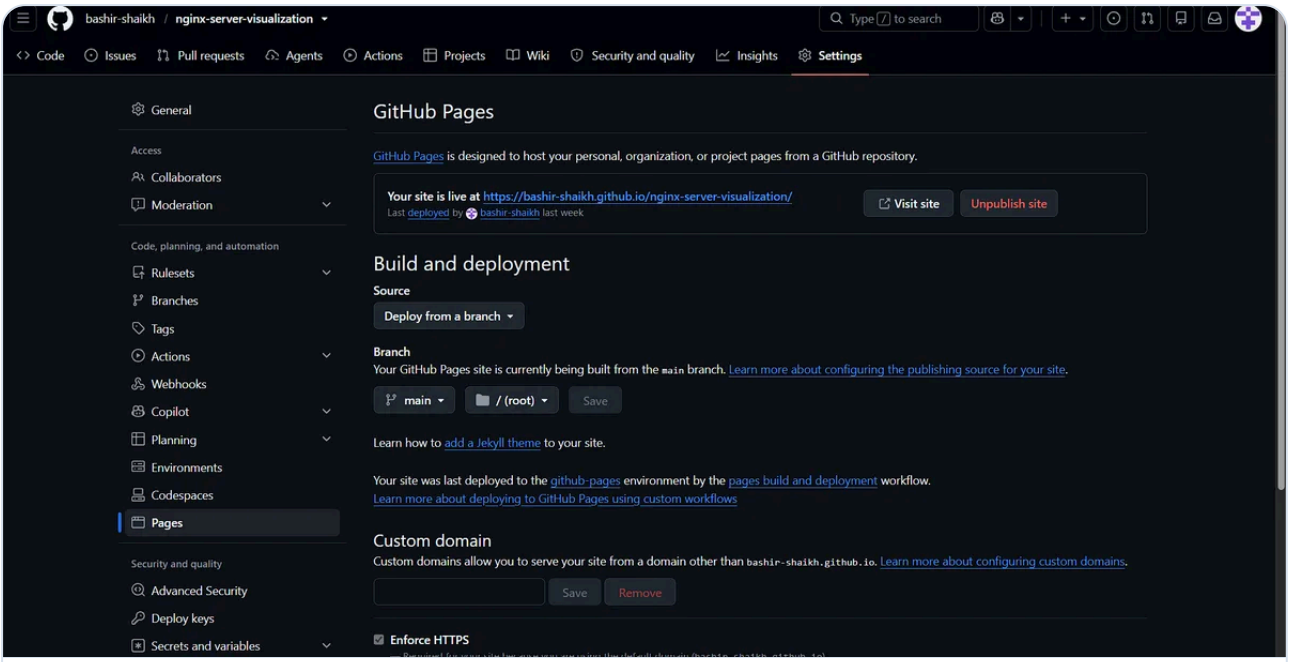


Figure 1: GitHub Pages settings for bashir-shaikh/nginx-server-visualization. This is the real settings screen for this project. What to look for:

- The banner says the site is live at `https://bashir-shaikh.github.io/nginx-server-visualization/`.
- Source is **Deploy from a branch**, publishing the `main` branch from the `/ (root)` folder.
- The site was deployed by the **pages build and deployment** workflow into the `github-pages` environment.
- No custom domain is set, and **Enforce HTTPS** is ticked.

Success looks like

Settings, Pages shows your branch and folder. The Actions tab lists a run named **pages build and deployment**.

If it fails: Branch dropdown is empty

The repository has no commits yet. Finish step 5.

7 Wait for deployment and find your URL

The address is how anyone reaches your site.

Open the **Actions** tab and wait for a green check on **pages build and deployment**. GitHub says changes can take up to about 10 minutes. Then return to **Settings, Pages**: your live address is shown at the top.

Repository name	Site address
`your-username.github.io`	`https://your-username.github.io/`
`nginx-verse-site`	`https://your-username.github.io/nginx-verse-site/`

Success looks like

The run shows Success and Settings, Pages shows “Your site is live”.

If it fails: Red X on the run

Open the run and read the log. Most failures are wrong branch, wrong folder, or a missing index.html.

8 Verify the live site

A green deployment does not prove your paths and styling work. Check content, styling, and status codes.

Open your site address, then confirm the heading, the paragraph, and the green header. Open browser tools with **F12**, go to **Network**, reload, and check that `style.css` returns **200**.

Optional command-line check

terminal Copy

```
curl -I https://your-username.github.io/nginx-verse-site/
```

expected output

```
HTTP/2 200
content-type: text/html; charset=utf-8
server: GitHub.com
```

`-I` requests headers only. `200` means success. `server: GitHub.com` confirms GitHub serves the site, not your own Nginx.

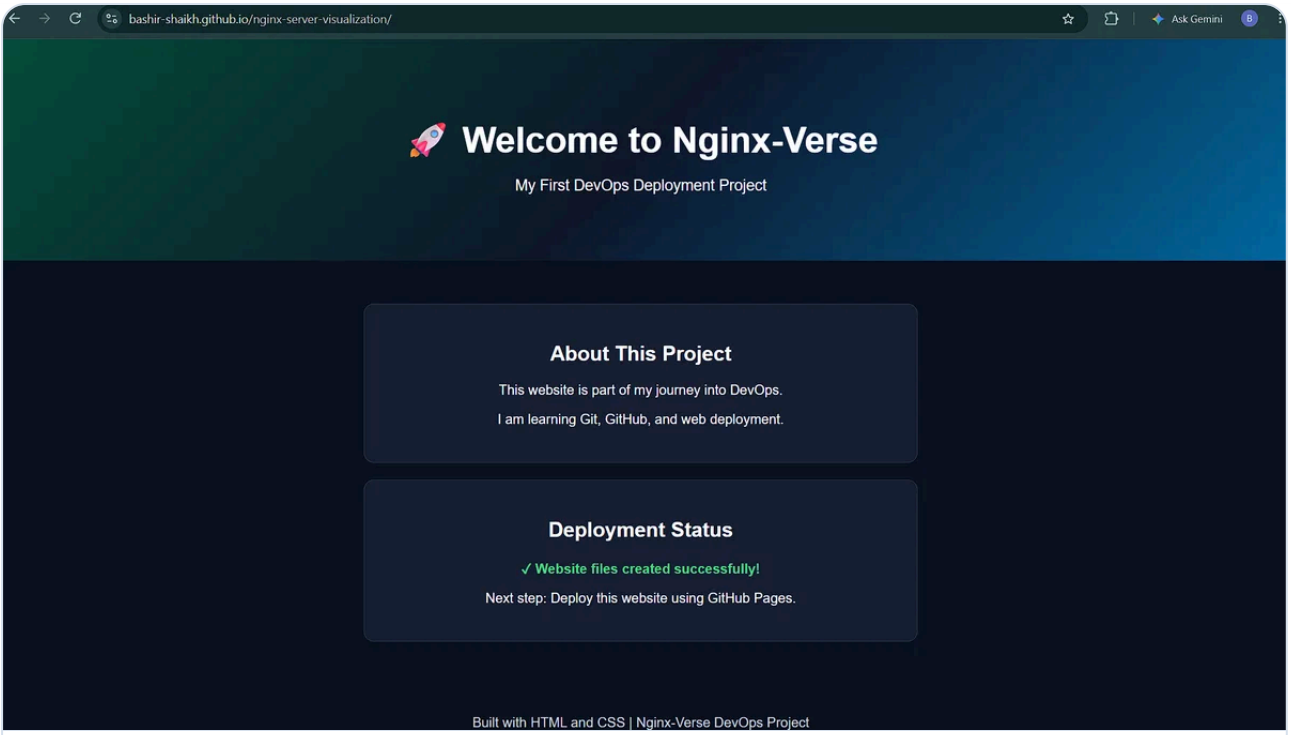


Figure 2: The live site in a browser. This is the real deployed site. What to look for:

- The address bar shows `bashir-shaikh.github.io/nginx-server-visualization/`, the GitHub Pages address from Figure 1.
- The HTML content loads: the “Welcome to Nginx-Verse” heading, the “About This Project” card and the “Deployment Status” card.
- The CSS loads too: a green-to-blue header gradient, a dark background and rounded cards. If the CSS path were wrong, the page would appear as plain unstyled text.
- The footer reads “Built with HTML and CSS | Nginx-Verse DevOps Project”.

Success looks like

HTTPS padlock shown, content visible, styling applied, `style.css` returns 200.

> **If it fails: Styling missing online but fine locally**
> Letter case differs (Style.css and style.css). GitHub’s servers are case-sensitive. Rename the file or the link to match exactly.

9 Update the site later

This is the loop you will use for every future change, and it is the manual version of what Jenkins automates in Project 3.

Edit a file, commit it, push it. Pages redeploys automatically.

terminal Copy

```
git add .
git commit -m "Update homepage text"
git push origin main
```

Watch the Actions tab for a new run, then hard refresh the site with Ctrl+F5.

Success looks like

Your change appears on the live site after the new run turns green.

> **If it fails: Old content still shows**
> Hard refresh, then confirm the new run finished.

When something goes wrong

These are common problems for this setup. Find your symptom, run the check, apply the fix.

Symptom	Likely cause	Check	Fix
404 at your site address	`index.html` missing, misnamed, or outside the published folder	Open the repository on the selected branch	Rename to exactly `index.html` and move it to the root
404 right after enabling Pages	Deployment still running	Actions tab shows a running job	Wait a few minutes, then press Ctrl+F5

Page loads without styling	Wrong CSS path or letter case	Browser tools, Network tab: `style.css` shows 404	Match the `href` to the real file path and case
Styling breaks on a project site	Absolute path such as `/css/style.css`	View page source	Use a relative path: `css/style.css`
Old content after an update	Browser cache or pending deployment	Actions tab	Hard refresh with Ctrl+F5
`Authentication failed` on push	GitHub does not accept account passwords for Git over HTTPS	Read the error text	Use a personal access token, Git Credential Manager, or SSH
`failed to push some refs`	The remote has commits you do not have	`git status`	`git pull origin main`, then push again
No Pages option in Settings	Not the owner, or private repository on a plan without Pages	Check repository visibility	Use a public repository you own

Security notes

Public means public

Everything in a public repository, including old commits, is visible to everyone. Never commit passwords, tokens or private data.

Protect your account

Turn on two-factor authentication. Give personal access tokens the smallest scope and an expiry date.

Keep HTTPS on

Leave “Enforce HTTPS” enabled in Pages settings so visitors’ traffic is encrypted.

Trust external code

Load third-party CSS or JavaScript only from sources you trust, because it runs on your page.

What you can say you learned

Static hosting

- Hosting versus running a server
- Why `index.html` is the entry point

Git and GitHub

- Repository, branch, commit, push
- Publishing a branch and folder

Web fundamentals

- HTML structure and CSS linking
- Relative paths and case sensitivity

Verification

- Reading HTTP status codes
- Using the browser Network tab

Final checklist

Tick each item as you finish it. Your progress is saved in this browser.

0 of 12 complete

- [ ] GitHub account ready and signed in
- [ ] Public repository nginx-verse-site created
- [ ] Project folder with index.html and css/style.css created
- [ ] HTML and CSS preview correctly on my computer
- [ ] Files uploaded or pushed to the repository
- [ ] index.html visible at the root of the main branch
- [ ] Pages source set to Deploy from a branch, main, / (root)
- [ ] pages build and deployment run is green
- [ ] Public address opens over HTTPS

- [ ] Heading, text and green header display correctly
- [ ] style.css returns status 200
- [ ] No passwords or tokens in the repository

Official references

GitHub Pages documentation · Publishing source · Fixing 404 errors · Git documentation · MDN web development